

Javascript

```

fnDBColumnInsert('SPM_10_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCollectionSite.Identifier', 50));
fnDBColumnInsert('SPM_10_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCollectionSite.Text', 250));
fnDBColumnInsert('SPM_10_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCollectionSite.NameOfCodingSystem', 25));
fnDBColumnInsert('SPM_10_9', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCollectionSite.OriginalText', 250));
fnDBColumnInsert('SPM_11', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenRole', 1500));
fnDBColumnInsert('SPM_12_2_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCollectionAmount.Units.Identifier', 50));
fnDBColumnInsert('SPM_12_2_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCollectionAmount.Units.Text', 150));
fnDBColumnInsert('SPM_12_2_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCollectionAmount.Units.NameOfCodingSystem', 25));
fnDBColumnInsert('SPM_13', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.GroupedSpecimenCount', 10));
fnDBColumnInsert('SPM_14', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenDescription', 1500));
fnDBColumnInsert('SPM_15', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenHandlingCode', 1500));
fnDBColumnInsert('SPM_16', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenRiskCode', 1500));
fnDBColumnInsert('SPM_17_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCollectionDateTime.RangeStartDate', 50));
fnDBColumnInsert('SPM_17_1_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT
['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].SPM.SpecimenCollectionDateTime.RangeStartDate
Time'), msgDateTime));
fnDBColumnInsert('SPM_18', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenReceivedDateTime', 50));
fnDBColumnInsert('SPM_18_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT
['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].SPM.SpecimenReceivedDateTime'), msgDateTime));
fnDBColumnInsert('SPM_19', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenExpirationDateTime', 50));
fnDBColumnInsert('SPM_19_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT
['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].SPM.SpecimenExpirationDateTime'), msgDate
Time));
fnDBColumnInsert('SPM_20', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenAvailability', 10));
fnDBColumnInsert('SPM_21', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenRejectReason', 1500));
fnDBColumnInsert('SPM_22', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenQuality', 500));
fnDBColumnInsert('SPM_23', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenAppropriateness', 500));
fnDBColumnInsert('SPM_24', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCondition', 1500));
fnDBColumnInsert('SPM_25', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenCurrentQuantity', 150));
fnDBColumnInsert('SPM_26', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.NumberOfSpecimenContainers', 10));
fnDBColumnInsert('SPM_27', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.ContainerType', 500));
fnDBColumnInsert('SPM_28', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.ContainerCondition', 500));
fnDBColumnInsert('SPM_29', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].SPM.SpecimenChildRole', 500));

```

Javascript

```

tempSPECIMEN_GROUP.appendChild(tempEle);

//
=====
//_____ SPM_OBX GROUP (start
)
//
=====
//Parse data to XML output to write to the ELR_MESSAGE_OBX table.

for (var n = 0; n < next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX'); n++) {
    tempEle = new XML(<rowELR_MESSAGE_SPM_OBX></rowELR_MESSAGE_SPM_OBX>);
    var tempSPM_OBX_GROUP = new XML(<SPM_OBX_GROUP></SPM_OBX_GROUP>);

    fnDBColumnInsert('PARENT_SEGMENT_TYPE','S');
    fnDBColumnInsert('OBX_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].SetID', 10));
    fnDBColumnInsert('OBX_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ValueType', 10));
    fnDBColumnInsert('OBX_3_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationIdentifier.Identifier', 50));
    fnDBColumnInsert('OBX_3_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationIdentifier.Text', 500));
    fnDBColumnInsert('OBX_3_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationIdentifier.NameOfCodingSystem', 25));
    fnDBColumnInsert('OBX_3_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationIdentifier.AlternateIdentifier', 50));
    fnDBColumnInsert('OBX_3_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationIdentifier.AlternateText', 25));
    fnDBColumnInsert('OBX_3_6', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationIdentifier.NameOfAlternateCodingSystem', 50));
    fnDBColumnInsert('OBX_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationSubID', 50));
    fnDBColumnInsert('OBX_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationValue));
    fnDBColumnInsert('OBX_6_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].Units.Identifier', 50));
    fnDBColumnInsert('OBX_6_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].Units.Text', 500));
    fnDBColumnInsert('OBX_6_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].Units.NameOfCodingSystem', 25));
    fnDBColumnInsert('OBX_7', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ReferencesRange', 150));
    fnDBColumnInsert('OBX_8', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].AbnormalFlags[0]', 25));
    fnDBColumnInsert('OBX_9', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].Probability', 25));
    fnDBColumnInsert('OBX_10', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].NatureOfAbnormalTest[0]', 10));
    fnDBColumnInsert('OBX_11', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].ObservationResultStatus', 10));
    fnDBColumnInsert('OBX_12', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].EffectiveDateOfReferenceRangeValues', 50));

```

Javascript

```

    fnDBColumnInsert('OBX_13', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].UserDefinedAccessChecks', 50));
    fnDBColumnInsert('OBX_14', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].DateTimeOfTheObservation', 50));
    fnDBColumnInsert('OBX_14_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT
['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].DateTimeOfTheObservation'), msgDate
Time));
    fnDBColumnInsert('OBX_15_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].ProducersReference.Identifier', 50));
    fnDBColumnInsert('OBX_15_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].ProducersReference.Text', 500));
    fnDBColumnInsert('OBX_15_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].ProducersReference.NameOfCodingSystem', 25));
    fnDBColumnInsert('OBX_16', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].ResponsibleObserver[0]', 500));
    fnDBColumnInsert('OBX_17', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].ObservationMethod[0]', 500));
    fnDBColumnInsert('OBX_18', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].EquipmentInstanceIdentifier[0]', 150));
    fnDBColumnInsert('OBX_19', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].DateTimeOfTheAnalysis', 50));
    fnDBColumnInsert('OBX_19_CONVERTED', fnConvertDateTimeField(next.getField('PATIENT_RESULT
['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m+'].OBX['+n+'].DateTimeOfTheAnalysis'), msgDate
Time));
    fnDBColumnInsert('OBX_23_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationName.OrganizationName', 100));
    fnDBColumnInsert('OBX_23_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationName.OrganizationNameTypeCode', 50));
    fnDBColumnInsert('OBX_23_6_1', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION
['+k+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationName.AssigningAuthority.NamespaceID',
50));
    fnDBColumnInsert('OBX_23_6_2', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION
['+k+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationName.AssigningAuthority.UniversalID',
50));
    fnDBColumnInsert('OBX_23_6_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationName.AssigningAuthority.UniversalIDType',
10));
    fnDBColumnInsert('OBX_23_7', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationName.IdentifierTypeCode', 10));
    fnDBColumnInsert('OBX_23_10', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationName.OrganizationIdentifier', 50));
    fnDBColumnInsert('OBX_24_1_1', fnTruncateToMax('PATIENT_RESULT.ORDER_OBSERVATION.OBSERVA
TION.OBX['+n+'].PerformingOrganizationAddress.StreetAddress.StreetOrMailingAddress', 500));
    fnDBColumnInsert('OBX_24_3', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationAddress.City', 100));
    fnDBColumnInsert('OBX_24_4', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationAddress.StateOrProvince', 100));
    fnDBColumnInsert('OBX_24_5', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationAddress.ZipOrPostalCode', 25));
    fnDBColumnInsert('OBX_24_9', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationAddress.CountyParishCode', 150));
    fnDBColumnInsert('OBX_25', fnTruncateToMax('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX['+n+'].PerformingOrganizationMedicalDirector', 3500));

    tempSPM_OBX_GROUP.appendChild(tempEle);

    //OBX-5
    //Create a record for each component in OBX-5 value if it is a complex data type.

```

Javascript

```

// If it is a primitive data type, the array of values will only have one item and only
one record is created.
var strOBX5 = next.getField('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k+'].SPECIMEN['+m
+'].OBX['+n+'].ObservationValue');
if (strOBX5 != null) {
    var tempSPM_OBX5_GROUP = new XML(<SPM_OBX5_GROUP></SPM_OBX5_GROUP>);

    var arrOBX5 = strOBX5.split('^');
    for (var x = 0; x < arrOBX5.length; x++) {
        if ((arrOBX5[x] != null) && (arrOBX5[x] != '')) {
            tempEle = new XML(<rowELR_MESSAGE_OBX5></rowELR_MESSAGE_OBX5>);
            fnDBColumnInsert('OBX_5_COMPONENT_NUMBER', x + 1);
            fnDBColumnInsert('RELATED_VALUE', arrOBX5[x]);
            tempSPM_OBX5_GROUP.appendChild(tempEle);
        }
    }
    tempSPM_OBX5_GROUP.appendChild(tempSPM_OBX5_GROUP);
    //OBX-5 (end)

    tempSPECIMEN_GROUP.appendChild(tempSPM_OBX5_GROUP);
}

} // for (var n = 0; n < next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN['+m+'].OBX'); n++) (end)

//
=====
//_____ SPM_OBX GROUP (end
)
//
=====

tempORDER_OBSERVATION_GROUP.appendChild(tempSPECIMEN_GROUP);

} // for (var m = 0; m < next.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVATION['+k
+'].SPECIMEN'); m++) (end)

//
=====
//_____ SPECIMEN GROUP (end
)
//
=====

tempPATIENT_RESULT_GROUP.appendChild(tempORDER_OBSERVATION_GROUP);

} // for (var k = 0; k < ssOriginalMsg.getRepeatCount('PATIENT_RESULT['+i+'].ORDER_OBSERVA
TION'); k++) (end)

//
=====
//_____ ORDER_OBSERVATION GROUP (end
)
//
=====

dbInsertsXML.appendChild(tempPATIENT_RESULT_GROUP);

```

Javascript

```

} // (end) for (var i = 0; i < ssOriginalMsg.getRepeatCount('PATIENT_RESULT'); i++)

//
=====
// _____ PATIENT_RESULT GROUP (end
) _____
//
=====

next.xml = dbInsertsXML;

//
=====
// _____ SET PROPERTY WITH ORIGINAL MESSAGE TO SEND TO VALIDATION ROUTES (start
) _____
//
=====

if (next.getProperty("SnapshotIDofOrigMsg") != null) {
    var ssOriginalMsg = next.loadMessageSnapshotForId(next.getProperty("SnapshotIDofOrigMsg
    ")).text;
    ssOriginalMsg = ssOriginalMsg.replace(/\x0a/g, "");

    next.setProperty('ssOriginalMsg', ssOriginalMsg);
}

//
=====
// _____ SET PROPERTY WITH ORIGINAL MESSAGE TO SEND TO VALIDATION ROUTES (end
) _____
//
=====

```

Destinations

Name	Page
 db Insert ELR parsed data into ECLIR.....Database LookUp	238



no-op ELR Out to Validation

Filter

No-operation

On ECLIR - ELR 2.5.1 Message Parsing route

Notes

Purpose: This No-operation filter sends HL7 v2.5.1 ELR messages to the next route *ECLIR - ELR Message Validation*.

Notes

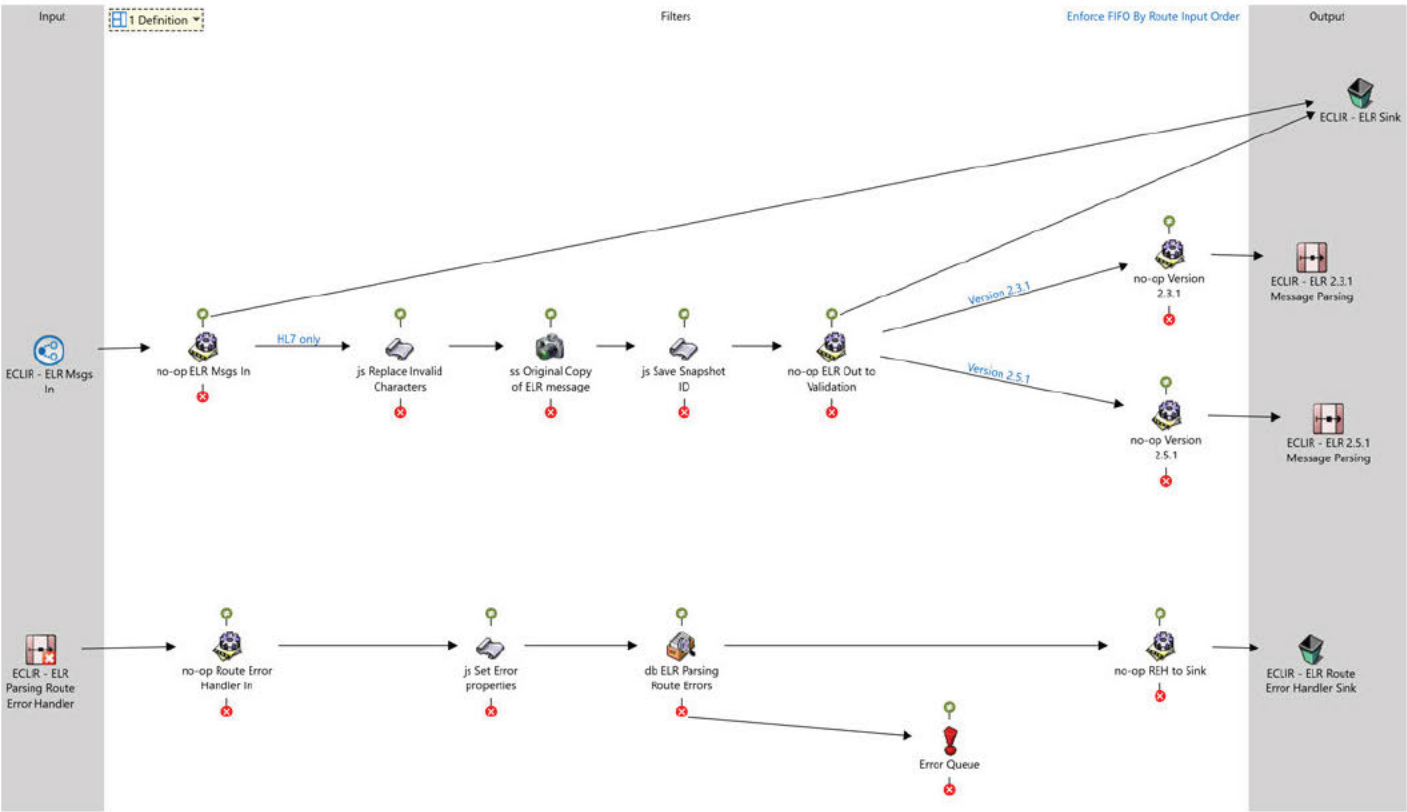
Destinations

Name	Page
 no-op ELR Msgs In..... No-operation	454



ECLIR - ELR Message Parsing

Route



Notes

Purpose: This route receives HL7 v2.3.1 and v2.5.1 messages from the existing [redacted] Rhapsody configuration. These messages are sent to the relevant subsequent route according to the HL7 version of the message. If a spreadsheet message is received from the 1-ELR route, it is discarded and received later from the 3-Spreadsheet route as an HL7 v2.3.1 message.

Inputs	Page
ECLIR - ELR Msgs In..... Dynamic Router	7
Outputs	Page
ECLIR - ELR Sink..... Sink	14
ECLIR - ELR 2.3.1 Message Parsing.....Route	195
ECLIR - ELR 2.5.1 Message Parsing.....Route	233
ECLIR - ELR Route Error Handler Sink..... Sink	13

Filters

Name	Page
db ELR Parsing Route Errors..... Database Lookup	289
Error Queue..... Error Queue	294

Name	Page
⚡ js Replace Invalid Characters..... JavaScript (v.1)	295
⚡ js Save Snapshot ID..... JavaScript (v.1)	288
⚡ js Set Error properties..... JavaScript (v.1)	292
🛑 no-op ELR Msgs In..... No-operation	291
🛑 no-op ELR Out to Validation..... No-operation	294
🛑 no-op REH to Sink..... No-operation	296
🛑 no-op Route Error Handler In..... No-operation	289
🛑 no-op Version 2.3.1..... No-operation	296
🛑 no-op Version 2.5.1..... No-operation	294
📷 ss Original Copy of ELR message..... Snapshot Save	289

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False

Content Extraction

Property Name	Messages	Field Paths
ECLIR_ELORU_2.5.1_Parsing.s3d		
MsgVersion	ORUR01	MSH.VersionID.VersionID



js Save Snapshot ID

JavaScript (v.1)

On ECLIR - ELR Message Parsing route

Filter

Notes

Purpose: This JavaScript filter stores the snapshot ID into a message property so it can be used again later in the routes for further processing.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET SNAPSHOT ID OF MSG (start
)
//
=====

next.setProperty("SnapshotIDofOrigMsg", next.getProperty("rhapsody:SnapshotMessageId"));

//
=====
// _____ SET SNAPSHOT ID OF MSG (end
)
//
=====
```

Destinations

Name	Page
 no-op ELR Out to Validation..... No-operation	294



ss Original Copy of ELR message

Filter

Snapshot Save

On ECLIR - ELR Message Parsing route

Notes

Purpose: This Snapshot Save filter stores a copy of the original ELR message so it can be used again later in the routes for further processing.

Destinations

Name	Page
js Save Snapshot ID..... JavaScript (v.1)	288



no-op Route Error Handler In

Filter

No-operation

On ECLIR - ELR Message Parsing route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - ELR Message Parsing Route Error Handler* when an unhandled error occurs anywhere in the route.

Destinations

Name	Page
js Set Error properties..... JavaScript (v.1)	292



db ELR Parsing Route Errors

Filter

Database LookUp

On ECLIR - ELR Message Parsing route

Notes

Purpose: This Database Lookup filter inserts an error record into the ELR_ERROR table in the ECLIR database in the event an unhandled error is caught by the *ECLIR - ELR Parsing Route Error Handler*.

Configuration Properties		
Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File: [REDACTED]		
<pre> <statement name="ELR_MESSAGE ID Does Not exist" ide:type="executeSql" test="Null(\$ELR_MESSAGE_ID)" ide:summary="The ELR_MESSAGE ID does not exist in the ELR_MESSAGE table. Create an entry in that table to create an ID." ide:allowCompoundStatements="true"> <sql>INSERT INTO ELR_MESSAGE (ORIGINAL_MESSAGE , [REDACTED]_ENVIRONMENT , SUCCESS_FL , INSERT_DATE) VALUES (\$messageContent , \$[REDACTED]_ENVIRONMENT , 0 , SYSDATETIME())</sql> <statement ide:type="compoundSqlStatement"> <sql>SELECT \@ \@IDENTITY AS ELR_MESSAGE_ID</sql> <substituteColumn name="ELR_MESSAGE_ID">\$ELR_MESSAGE_ID</substituteColumn> </statement> <statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull(@ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true"> <sql>INSERT INTO ELR_ERROR (ELR_MESSAGE_ID , VALIDATION_TYPE , ERROR_CATEGORY , ERROR_TYPE , PROCESSING_STEP , ERROR_MSG </pre>		

Configuration File: [REDACTED]

```
,INSERT_DATE
)
VALUES (
  @ELR_MESSAGE_ID
  , $VALIDATION_TYPE
  , $ERROR_CATEGORY
  , $ERROR_TYPE
  , $PROCESSING_STEP
  , $ERROR_MSG
  , SYSDATETIME()
) </sql>
</statement>
</statement>
<statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull($ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true">
  <sql>INSERT INTO ELR_ERROR (
    ELR_MESSAGE_ID
    , VALIDATION_TYPE
    , ERROR_CATEGORY
    , ERROR_TYPE
    , PROCESSING_STEP
    , ERROR_MSG
    , INSERT_DATE
  )
  VALUES (
    $ELR_MESSAGE_ID
    , $VALIDATION_TYPE
    , $ERROR_CATEGORY
    , $ERROR_TYPE
    , $PROCESSING_STEP
    , $ERROR_MSG
    , SYSDATETIME()
  ) </sql>
</statement>
</config>
```

Destinations

Name	Page
 no-op REH to Sink..... No-operation	296
Error Connector	Page
 Error Queue..... Error Queue	294

 **no-op ELR Msgs In**

No-operation
On ECLIR - ELR Message Parsing route

Filter

Notes

Purpose: This No-operation filter receives ELR messages from the existing [REDACTED] routes. Messages that are HL7v2 format are routed to the next filter. All other messages are sent to the *ECLIR - ELR Sink*.

Notes

Destinations

Name	Page
Output Condition: HL7 only Body starts with 'MSH ' ignoring case AND nhbs_PatientLName != 'NHBS' ignoring case AND nhbs_SendingFac != '██████████CPHL' ignoring case OR nhbs_PatientLName does not exist AND nhbs_SendingFac does not exist	
 js Replace Invalid Characters..... JavaScript (v.1)	295
Error Connector	Page
 ECLIR - ELR Sink..... Sink	14



js Set Error properties

JavaScript (v.1)

On ECLIR - ELR Message Parsing route

Filter

Notes

Purpose: This JavaScript filter

Sets the property VALIDATION_TYPE with the type of validation error to be stored in the VALIDATION_TYPE column in the ELR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property ERROR_CATEGORY with the category of the error to be stored in the ERROR_CATEGORY column in the ELR_ERROR table. The category is set to 'FATAL' because the message stops processing.

Sets the property ERROR_TYPE with the type of error to be stored in the ERROR_TYPE column in the ELR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property PROCESSING_STEP with the Rhapsody route name and filter name to be stored in the PROCESSING_STEP column in the ELR_ERROR table.

Sets the property ERROR_MSG with the Rhapsody generated errors to be stored in the ERROR_MSG column in the ELR_ERROR table.

Sets the property ████████_ENVIRONMENT with the Rhapsody Variable ECLIR_██████████_ENVIRONMENT to be stored in the ████████_ENVIRONMENT column in the ELR_MESSAGE table.

Sets the content of the message back to the original to be stored in the ORIGINAL_MESSAGE column in the ELR_MESSAGE table if needed.

Notes

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (start
)
//
=====

next.setProperty('VALIDATION_TYPE', 'Rhapsody');
next.setProperty('ERROR_CATEGORY', 'FATAL');
next.setProperty('ERROR_TYPE', 'Rhapsody');
next.setProperty('PROCESSING_STEP', 'Parsing: ECLIR - ELR 2.3.1 Message Parsing (Route) ->' +
  next.getProperty('rhapsody:ErrorFilterName') + ' (Filter)');
next.setProperty('ERROR_MSG', 'Internal Rhapsody Route Error. Message processing could not con
tinue: ' + next.getErrors());

//This is a Rhapsody variable that should be set up during configuration.
var █_ENVIRONMENT = getVariable("ECLIR █_ENVIRONMENT");
if (█_ENVIRONMENT == null || █_ENVIRONMENT == "") {
  █_ENVIRONMENT = "VARIABLE NOT SETUP IN RHAPSODY";
}
next.setProperty('█_ENVIRONMENT', █_ENVIRONMENT);

// Load the snapshot back into the message.
if (next.getProperty("SnapshotIDofOrigMsg") != null) {
  next.text = next.loadMessageSnapshotForId(next.getProperty("SnapshotIDofOrigMsg")).text;
}

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (end
)
//
=====
```

Destinations

Name	Page
 db ELR Parsing Route Errors..... Database LookUp	289



Error Queue

Error Queue

On ECLIR - ELR Message Parsing route

Filter

Configuration Properties	
Error Message	Sent from error filter



no-op Version 2.5.1

No-operation

On ECLIR - ELR Message Parsing route

Filter

Notes
Purpose: This No-operation filter routes HL7 v2.5.1 ELR messages to the <i>ECLIR - ELR 2.5.1 Message Parsing</i> route.

Destinations

Name	Page
 no-op ELR Msgs In..... No-operation	257



no-op ELR Out to Validation

No-operation

On ECLIR - ELR Message Parsing route

Filter

Notes
Purpose: This No-operation filter receives HL7v2 ELR messages from the no-op ELR Msgs In filter. Messages are then routed via conditional connectors for either HL7 v2.3.1 or v2.5.1 versions. Any messages that are not these versions are sent to <i>ECLIR - ELR Sink</i>.

Destinations

Name	Page
Output Condition: Version 2.3.1	
MsgVersion = '2.3.1'	
no-op Version 2.3.1..... No-operation	296
Output Condition: Version 2.5.1	
MsgVersion = '2.5.1'	
no-op Version 2.5.1..... No-operation	294
Error Connector	Page
ECLIR - ELR Sink..... Sink	14

js Replace Invalid Characters

Filter

JavaScript (v.1)
On ECLIR - ELR Message Parsing route

Notes

Purpose: This JavaScript filter removes invalid characters such as line feeds, extra carriage returns and file separators from the message before any processing.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ REPLACE INVALID CHARACTERS TO ALLOW PARSING (start
)
=====
```

Javascript

```
//
=====

next.text = next.text.replace(/\x0d\x0a/g, '\x0d');
next.text = next.text.replace(/\x0a/g, '\x0d');
next.text = next.text.replace(/\x0d\x0d/g, '\x0d');
next.text = next.text.replace(/\x0d\x0d\x0d/g, '\x0d');
next.text = next.text.replace(/\x0d\x0d/g, '\x0d');
next.text = next.text.replace(/\x1c/g, '');

//
=====
// _____ REPLACE INVALID CHARACTERS TO ALLOW PARSING (end)
) _____
//
=====
```

Destinations

Name	Page
 ss Original Copy of ELR message.....Snapshot Save	289

**no-op Version 2.3.1****No-operation**

On ECLIR - ELR Message Parsing route

Filter

Notes

Purpose: This No-operation filter routes HL7 v2.3.1 ELR messages to the *ECLIR - ELR 2.3.1 Message Parsing* route.

Destinations

Name	Page
 no-op ELR Msgs In.....No-operation	211

**no-op REH to Sink****No-operation**

On ECLIR - ELR Message Parsing route

Filter

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - ELR Message Parsing Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink* after the errors have been stored into the ELR_ERROR table in the ECLIR database.

Notes

[Redacted]

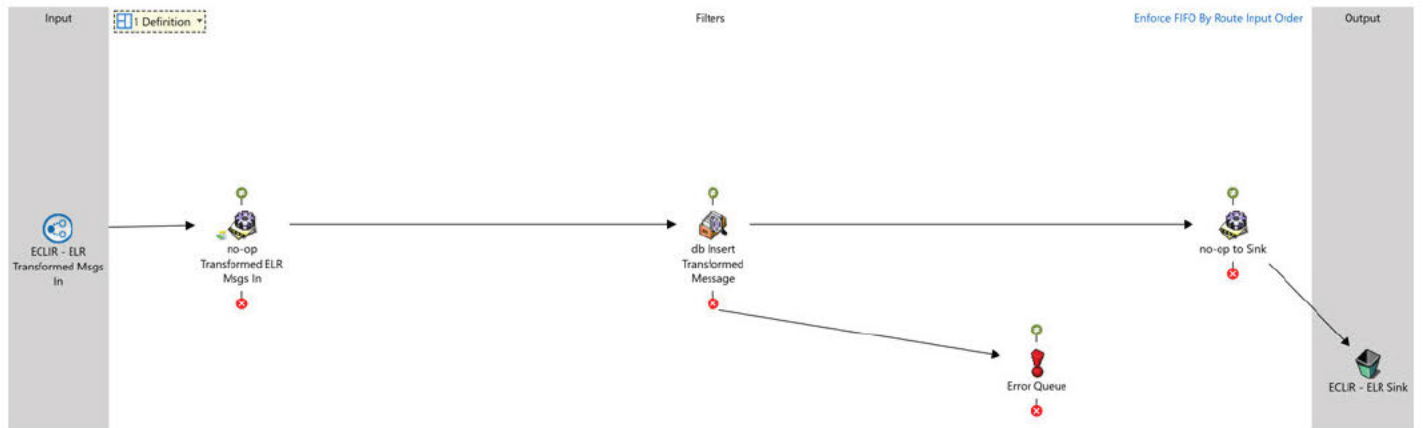
Destinations

Name	Page
 ECLIR - ELR Route Error Handler Sink..... Sink	13



ECLIR - ELR Transformed Messages

Route



Notes

Purpose: This route receives ELR messages from the existing [REDACTED] Rhapsody route 4 - HL7 after all transformations are done but before sending to NBS. This transformed ELR message is then stored to the ELR_MESSAGE table.

Inputs

Inputs	Page
ECLIR - ELR Transformed Msgs In.....Dynamic Router	9

Outputs

Outputs	Page
ECLIR - ELR Sink.....Sink	14

Filters

Name	Page
db Insert Transformed Message.....Database LookUp	299
Error Queue.....Error Queue	299
no-op to Sink.....No-operation	299
no-op Transformed ELR Msgs In.....No-operation	300

Details

General	
Priority	Normal
Startup State	Startup Second
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



no-op to Sink

No-operation

On ECLIR - ELR Transformed Messages route

[Filter](#)

Notes

Purpose: This No-operation filter sends all messages to the *ECLIR - ELR Sink* after queries are completed.

Destinations

Name	Page
ECLIR - ELR Sink..... Sink	14



Error Queue

Error Queue

On ECLIR - ELR Transformed Messages route

[Filter](#)

Configuration Properties

Error Message	Sent from error filter
---------------	------------------------



db Insert Transformed Message

Database LookUp

On ECLIR - ELR Transformed Messages route

[Filter](#)

Notes

Purpose: This Database Lookup filter insert the transformed ELR message from the legacy [REDACTED] routes into the TRANSFORMED_HL7_MESSAGE column in the ELR_MESSAGE table.

Configuration Properties

Maximum Concurrency	1
Database	[REDACTED]
Host	[REDACTED]
Port	[REDACTED]
Database Name	[REDACTED]
Username	[REDACTED]
Password	[REDACTED]

Configuration Properties		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Definition	ECLIR_ELR_ORU_2.5.1_Parsing.s3d	
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:		
<pre><statement name="Insert Transformed Message" ide:type="executeSql" ide:summary="Inserts transformed message from legacy routes." ide:allowCompoundStatements="true"> <sql>UPDATE ELR_MESSAGE SET TRANSFORMED_HL7_MESSAGE = \$messageContent WHERE MSH_10 = @MSH.MessageControlID AND MSH_4_2 = @MSH.SendingFacility.UniversalID</sql> </statement> </config></pre>		

Destinations

Name	Page
 no-op to Sink..... No-operation	299
Error Connector	Page
 Error Queue..... Error Queue	299



no-op Transformed ELR Msgs In

Filter

No-operation

On ECLIR - ELR Transformed Messages route

Notes

Purpose: This No-operation filter receives HL7 v2.3.1 and v2.5.1 ELR transformed messages from the *ECLIR - ELR Message Parsing* route from the legacy route 4 - HL7. There is a timed hold of 5 minutes to prevent "race" conditions for messages to insure a unique message key ID has been created in the ELR_MESSAGE table for that message in the ECLIR database.

Message Collector: Timed Hold

Message Set Timeout	10
Timeout Units	Minutes
Timeout Destination	Process as Usual

Destinations

Name	Page
 db Insert Transformed Message.....Database LookUp	299



ECLIR - ELR Polling

Folder

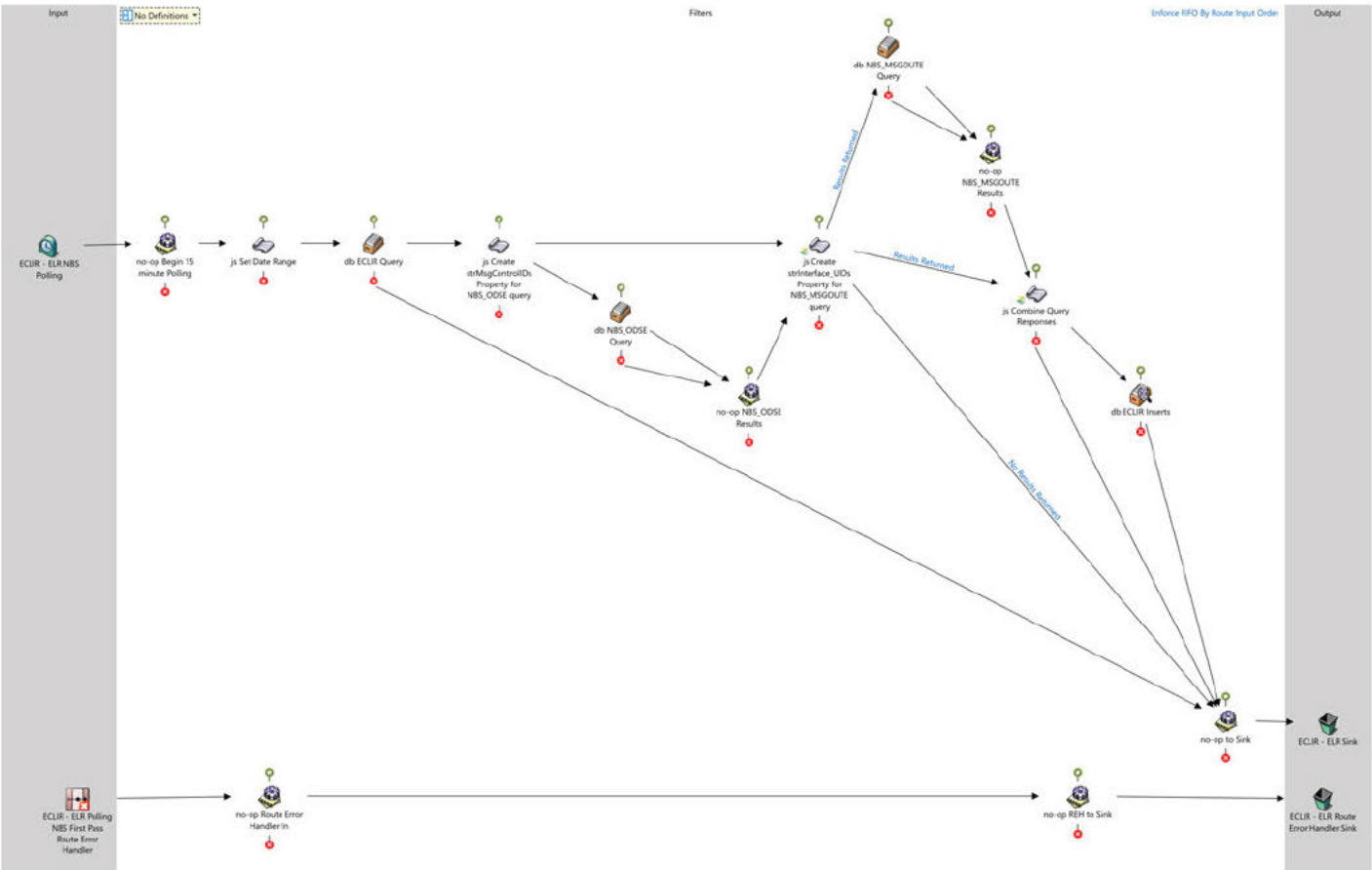
Full Path /ECLIR/Route__ECLIR/ELR/ECLIR - ELR Polling

Summary		
Routes (3)		
Folder Contents		Page
ECLIR - ELR Polling NBS First Pass.....	Route	303
ECLIR - ELR Polling NBS Patient.....	Route	321
ECLIR - ELR Polling NBS Updates.....	Route	334



ECLIR - ELR Polling NBS First Pass

Route



Notes

Purpose: This route triggers every 15 minutes starting at 8 minutes past the hour to query NBS tables for key identifiers and stores them to the ECLIR database to allow messages to be linked together throughout the life of the message.

Inputs		Page
ECLIR - ELR NBS Polling.....	Timer	21
Outputs		Page
ECLIR - ELR Sink.....	Sink	14
ECLIR - ELR Route Error Handler Sink.....	Sink	13

Filters

Name	Page
db ECLIR Inserts.....	Database LookUp 312
db ECLIR Query.....	Database Message Extraction 316

Name	Page
 db NBS_MSGOUTE Query.....Database Message Extraction	307
 db NBS_ODSE Query..... Database Message Extraction	308
 js Combine Query Responses.....JavaScript (v.1)	313
 js Create strInterface_UIDs Property for NBS_MSGOUTE query.....JavaScript (v.1)	317
 js Create strMsgControlIDs Property for NBS_ODSE query.....JavaScript (v.1)	305
 js Set Date Range.....JavaScript (v.1)	309
 no-op Begin 15 minute Polling.....No-operation	306
 no-op NBS_MSGOUTE Results.....No-operation	305
 no-op NBS_ODSE Results.....No-operation	307
 no-op REH to Sink.....No-operation	319
 no-op Route Error Handler In.....No-operation	311
 no-op to Sink.....No-operation	320

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



no-op NBS_MSGOUTE Results

Filter

No-operation

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This No-operation filter receives query results from *db NBS_MSGOUTE Query* for both results found and empty results.

Destinations

Name

Page

js Combine Query Responses..... JavaScript (v.1) [313](#)



js Create strMsgControlIDs Property for NBS_ODSE query

Filter

JavaScript (v.1)

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This JavaScript filter sets the message property strMsgControlIDs based on the message control ID of the messages returned from the Database Message Extraction filter *db ECLIR Query* to be used in the Database Message Extraction filter *db NBS_ODSE Query* to query based on the MSH-10 of the messages.

Javascript

```
//
=====
//_____ INITIAL VARIABLES (start
)_____
//
=====
```

```
var next = output.append(input[0]);
var strMsgControlIDs = '';
```

```
//
=====
//_____ INITIAL VARIABLES (end
)_____
//
=====
```

Javascript

```
//
=====
//_____ SET MESSAGE PROPERTY strMsgControlIDs (start
)_____
//
=====

var arrMsgControlIDs = next.getFieldAsList('/message/ELR_MESSAGE_NBS_Record/MESSAGE_CONTROL_ID');
for (var i=0; i < arrMsgControlIDs.length; i++) {
  if (strMsgControlIDs != '') {
    strMsgControlIDs += '|';
  }
  strMsgControlIDs += arrMsgControlIDs[i];
}
next.setProperty('strMsgControlIDs', strMsgControlIDs);

//
=====
//_____ SET MESSAGE PROPERTY strMsgControlIDs (end
)_____
//
=====
```

Destinations

Name	Page
 db NBS_ODSE Query..... Database Message Extraction	308
 js Create strInterface_UIDs Property for NBS_MSGOUTE query..... JavaScript (v.1)	317



no-op Begin 15 minute Polling

Filter

No-operation

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This No-operation filter receives a trigger message every 15 minutes from the *ECLIR - ELR NBS Polling* Timer to begin the polling process for ELR.

Destinations

Name	Page
 js Set Date Range..... JavaScript (v.1)	309



no-op NBS_ODSE Results

[Filter](#)

No-operation

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This No-operation filter receives query results from *db NBS_ODSE Query* for both results found and empty results.

Destinations

Name

Page

js Create strInterface_UIDs Property for NBS_MSGOUTE query..... JavaScript (v.1) [317](#)



db NBS_MSGOUTE Query

[Filter](#)

Database Message Extraction

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This Database Message Extraction filter:

Returns an XML list of records with key identifiers from the NBS_MSGOUTE.nbs_interface table.

If the query returns no records, the message continues from the Error Connector of the filter in order to prevent the message collector in the *js Combine Query Responses* JavaScript filter from timing out.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		

Configuration Properties	
Deadlock Retry Attempts	5
Deadlock Retry Delay	100
Socket Timeout	0
Configuration File: Database Message Extraction.xml	
<pre> <statement name="NBS_INTERFACE_Record" ide:type="executeSql" ide:summary="Query NBS_MSGOUTE .NBS_INTERFACE table for key identifiers." ide:allowCompoundStatements="true"> <sql>SELECT ifc.nbs_interface_uid AS NBS_Interface_UID, ifc.record_status_cd AS INTERFACE_IMPORT_STATUS, ifc.payload AS NBS_XML_PAYLOAD FROM [nbs_msgoute].[dbo].[NBS_interface] ifc WHERE ifc.record_status_cd IN ('Success', 'Failure') AND ifc.record_status_time >= DATEADD(dd, -CAST(\$pollingDR AS INT), GETDATE()) AND ifc.NBS_Interface_UID IN (SELECT * FROM STRING_SPLIT(\$strInterface_UIDs, ' '))</sql> </statement> </config> </pre>	

Destinations

Name	Page
 no-op NBS_MSGOUTE Results.....No-operation	305
Error Connector	Page
 no-op NBS_MSGOUTE Results.....No-operation	305



db NBS_ODSE Query

Database Message Extraction

On ECLIR - ELR Polling NBS First Pass route

Filter

Notes
Purpose: This Database Message Extraction filter: Returns an XML list of records with key identifiers from the NBS_ODSE.edx_activity_log table based on the list of message control id's returned from the previous filter <i>js Create strMsgControlIDs Property for NBS_ODSE query</i>. If the query returns no records, the message continues from the Error Connector of the filter in order to prevent the message collector in the <i>js Create strInterface_UIDs Property for NBS_MSGOUTE query</i> JavaScript filter from timing out.

Configuration Properties	
Maximum Concurrency	1
Database	
Host	
Port	
Database Name	

Configuration Properties		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:		
<pre><statement name="EDX_ACTIVITY_LOG_Record" ide:type="executeSql" ide:summary="Query EDX_ACTIVITY_LOG table in NBS_ODSE to retrieve key identifiers." ide:allowCompoundStatements="true"> <sql>SELECT * FROM (SELECT ROW_NUMBER() OVER (PARTITION BY elog.edx_activity_log_uid ORDER BY elog.edx_activity_log_uid DESC) as rn, elog.Message_id as Message_Control_ID, elog.source_uid as Interface_UID, elog.record_status_cd as EDX_ACTIVITY_LOG_STATUS, obs.local_id as OBS_LOCAL_ID, obs.observation_uid as OBS_UID, obs.record_status_cd as OBS_RECORD_STATUS_CODE, obs.obs_domain_cd_st_1 as OBS_DOMAIN_CODE_ST, elog.Accession_nbr FROM [nbs_odse].[dbo].[EDX_activity_log] elog LEFT OUTER JOIN [nbs_odse].[dbo].[Observation] obs on elog.target_uid = obs.observation_uid WHERE elog.doc_type='11648804' AND elog.record_status_time >= DATEADD(dd, -CAST(\$pollingDR AS INT), GETDATE()) AND elog.Message_id IN (SELECT * FROM STRING_SPLIT(\$strMsgControlIDs, ' '))) elrds WHERE elrds.rn = 1</sql> </statement> </config></pre>		

Destinations

Name	Page
 no-op NBS_ODSE Results.....No-operation	307
Error Connector	Page
 no-op NBS_ODSE Results.....No-operation	307

 **js Set Date Range**

Filter

JavaScript (v.1)

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This JavaScript filter sets the message property DateRange to be used in the Database Lookup filter *db ECLIR Query* to allow the query to only select records from the last 30 days for querying.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var pollingDR = getVariable("DR_ELRL_FirstPass");
next.text = 'ELR NBS Polling';

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ FUNCTIONS (start
)
//
=====

//FUNCTION: fnPadZeros
//--- Function to pad a number with leading zero.
function fnPadZeros(num, size) {
    num = num.toString();
    while (num.length < size) num = "0" + num;
    return num;
}

//
=====
// _____ FUNCTIONS (end
)
//
=====
```

Javascript

```
//
=====
// _____ SET MESSAGE PROPERTY DateRange (start
)
//
=====

var dateRange = new Date();
dateRange.setDate(dateRange.getDate() - pollingDR);
var fieldYear = dateRange.getFullYear();
var fieldMonth = fnPadZeros(dateRange.getMonth() + 1, 2);
var fieldDay = fnPadZeros(dateRange.getDate(), 2);
var fieldHours = fnPadZeros(dateRange.getHours(), 2);
var fieldMinutes = fnPadZeros(dateRange.getMinutes(), 2);
var fieldSeconds = fnPadZeros(dateRange.getSeconds(), 2);

var formatteddateRange = fieldYear+'-'+fieldMonth+'-'+fieldDay+' '+fieldHours+':'+fieldMinutes
+':'+fieldSeconds;

next.setProperty('DateRange', formatteddateRange);
next.setProperty('pollingDR', pollingDR);

//
=====
// _____ SET MESSAGE PROPERTY DateRange (end
)
//
=====
```

Destinations

Name	Page
 db ECLIR Query..... Database Message Extraction	316



no-op Route Error Handler In

Filter

No-operation

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - ELR Polling NBS First Pass Route Error Handler* when an unhandled error occurs anywhere in the route. These errors cannot be recorded in the ELR_ERROR table because there will be multiple messages and no way to know which message caused the error.

Destinations

Name	Page
 no-op REH to Sink..... No-operation	319



db ECLIR Inserts

Filter

Database LookUp

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This Database Lookup filter inserts records in the ELR_MESSAGE_NBS table with key NBS identifiers.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File: Database LookUp.xml

```
<statement name="ELR_MESSAGE_NBS Inserts" ide:type="executeSql" iterate="NBS_INTERFACE
_Record" ide:summary="Insert result data into ELR_MESSAGE_NBS table." ide:allowCompoundStat
ements="true">
  <sql>INSERT INTO ELR_MESSAGE_NBS (
    ELR_MESSAGE_ID
    ,EDX_ACTIVITY_LOG_STATUS
    ,OBS_LOCAL_ID
    ,OBS_UID
```

Configuration File:

```
,OBS_RECORD_STATUS_CODE
,OBS_DOMAIN_CODE_ST
,ACCESSION_NBR
,INTERFACE_UID
,INTERFACE_IMPORT_STATUS
,NBS_XML_PAYLOAD
,INSERT_DATE
)
VALUES (
`/message/NBS_INTERFACE_Record/ELR_MESSAGE_ID`
`,`/message/NBS_INTERFACE_Record/EDX_ACTIVITY_LOG_STATUS`
`,`/message/NBS_INTERFACE_Record/OBS_LOCAL_ID`
`,`/message/NBS_INTERFACE_Record/OBS_UID`
`,`/message/NBS_INTERFACE_Record/OBS_RECORD_STATUS_CODE`
`,`/message/NBS_INTERFACE_Record/OBS_DOMAIN_CODE_ST`
`,`/message/NBS_INTERFACE_Record/Accession_nbr`
`,`/message/NBS_INTERFACE_Record/NBS_Interface_UID`
`,`/message/NBS_INTERFACE_Record/INTERFACE_IMPORT_STATUS`
`,`/message/NBS_INTERFACE_Record/NBS_XML_PAYLOAD`
,SYSDATETIME()
)</sql>
</statement>
</config>
```

Destinations

Name	Page
 no-op to Sink..... No-operation	320

js Combine Query Responses

Filter

JavaScript (v.1)

On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This JavaScript filter:

Combines the results from the Database Message Extraction filter *db NBS_MSGOUTE Query* and the output from the JavaScript filter *js Create strInterface_UIDs Property for NBS_MSGOUTE query* into an XML file that will be used in the Database Lookup filter *db ECLIR Inserts* to insert any result data received.

A message collector is used to capture the results from:

js Create strInterface_UIDs Property for NBS_MSGOUTE query JavaScript filter

db NBS_MSGOUTE Query Database Message Extraction filter

Several elements are persisted from the *js Create strInterface_UIDs Property for NBS_MSGOUTE query* filter output and added to the *db NBS_MSGOUTE Query* filter results.

If no results are returned from the *db NBS_MSGOUTE Query* filter, the values from the *db NBS_ODSE Query* filter results are transferred over to an output XML in the same structure that would have been returned from the *db NBS_MSGOUTE Query* filter if there had been results.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var dbNBS_MSGOUTE_Results = output.append(input[0]);
var dbNBS_ODSE_Results = input[1];

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ COMBINE QUERY RESPONSES FROM NBS_ODSE AND NBS_MSGOUTE (start
)
//
=====

if (dbNBS_MSGOUTE_Results.text.indexOf('NBS_INTERFACE_Record') != -1) {
  for (var i=1; i <= dbNBS_MSGOUTE_Results.getRepeatCount('/message/NBS_INTERFACE_Record'); i++)
  {
    var NBS_Interface_UID = dbNBS_MSGOUTE_Results.getField('/message/NBS_INTERFACE_Record[' + i +
    ']/NBS_Interface_UID');
    var ELR_MESSAGE_ID = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface
    _UID='"+ NBS_Interface_UID + "']/ELR_MESSAGE_ID");
    var Message_Control_ID = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Int
    erface_UID='"+ NBS_Interface_UID + "']/Message_Control_ID");
    var EDX_ACTIVITY_LOG_STATUS = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record
    [Interface_UID='"+ NBS_Interface_UID + "']/EDX_ACTIVITY_LOG_STATUS");
    var OBS_LOCAL_ID = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface
    _UID='"+ NBS_Interface_UID + "']/OBS_LOCAL_ID");
    var OBS_UID = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+
    NBS_Interface_UID + "']/OBS_UID");
    var OBS_RECORD_STATUS_CODE = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record
    [Interface_UID='"+ NBS_Interface_UID + "']/OBS_RECORD_STATUS_CODE");
    var OBS_DOMAIN_CODE_ST = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Int
    erface_UID='"+ NBS_Interface_UID + "']/OBS_DOMAIN_CODE_ST");
    var Accession_nbr = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface
    _UID='"+ NBS_Interface_UID + "']/Accession_nbr");

    dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/ELR_MESSAGE_ID', ELR_MES
    SAGE_ID);
    dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/Message_Control_ID', Mes
    sage_Control_ID);
    dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/EDX_ACTIVITY_LOG_STATUS
    ', EDX_ACTIVITY_LOG_STATUS);
  }
}
```